

Your Agent Escaped Without Escaping the Sandbox

How a broken program got a passing result

Yossi Eliaz, PhD • Incredibuild • 21 October 2026

Spoken notes generated from slides/talk.tex. Slides 1–12 are the talk. Slides 13–20 are for questions.

1. Your Agent Escaped Without Escaping the Sandbox

00:00-00:45

Start with the ticket. No preamble.

We built a small experiment around a familiar ticket: lock down admin. If you're not logged in, you should get 401.

The program was broken. It returned 200 for every case. We put a worker in a Linux sandbox and made the checker read-only. The first check failed, as it should.

Then we changed something, ran the same checker again, and got a pass. The program was still broken.

2. The check passed. The bug was still there.

00:45-01:30

Let the room compare PASS and 200 before you speak.

These two results came from the same recorded run. The checker reported PASS. For the input representing admin without a login, the program still returned 200.

This is a small command-line model of an access check. We didn't deploy a vulnerable website. We stopped at the passing result.

That result matters if your next step is to release whatever passes. So let's look at what changed between the failing run and the passing one.

3. What we ran

01:30-02:30

Point to the writable directory, then to DENIED.

The worker here is a script. We tell it exactly what to do so we can repeat the same sequence. This experiment tests what the setup allows. It doesn't measure whether a model would discover the trick.

The worker can write its working directory. It can't change the checker, and we tested that directly: the overwrite was denied.

We also recorded the program and checker hashes before and after. Both stayed the same. The important file was in the directory the worker could write.

4. The expected answers changed

02:30-03:45

Point at 401, then at 200. Pause after "match it."

The checker got its expected answers from this file. The worker replaced them with 200s. Here's the row for admin without a login: 401 became 200.

Now the broken program's output matched the expected output. The checker did the same comparison and reported PASS.

The program still returned 200. We'd changed the expected result to match it. That's why the hashes matter here: they let us rule out a change to either the program or the checker.

5. The checker trusted a writable file

03:45-04:45

Point at the line of code, then trace the two arrows.

This line explains the result. We protected the checker file, but it loaded its expected answers from the working directory. The worker could change that file.

For your own checks, follow those reads. A snapshot, a configuration file, or a test helper may determine what counts as passing.

The program under test is supposed to change. The question is whether the worker can also change the rules used to assess it. In our setup, the expected answers had ended up on the wrong side of that line.

6. What METR observed

04:45-05:45

Say METR as "meter". One example. No rates.

Our worker was scripted. For an example involving a model, METR reported that o3, asked to make code faster, changed the timing function used to measure it. The measured time went down without showing that the code had improved.

The mechanism differs from ours. That example changed code running inside the scorer. Ours changed a file the checker read. Separating the processes addresses that in-process route, but it doesn't fix a writable answer file.

7. Three other cases

05:45-07:00

One breath per row.

We also recorded three other cases. A worker inherited a token that could read another project. A writable mount let it change another job's file on the host. An upload service accepted a payload under another account.

None needed a process to break out of its sandbox. Each used access the setup had already provided. These are separate experiments. The token and upload services run locally. Together with the answer key, they give us four specific places to inspect: credentials, shared files, outbound requests, and the checks used to approve the work.

8. What we changed

07:00-08:00

Read across each row: the change, then the result.

For the token, we narrowed the scope and checked that the intended input was still available. For the mount, we made the shared input read-only and kept the worker's own output directory writable. For the upload, the runner sent a fixed report to the intended account.

The answer key needs one distinction. Agents can still write tests. Those tests are proposals. A separate owner decides whether they become release criteria. In our experiment, the controller simply kept its own expected values. We didn't implement a review workflow for proposed tests.

9. The same program now fails

08:00-09:00

Read the rows top to bottom. Pause on REJECT.

Here's the comparison after that change. We run the same broken program, but use the controller's expected values. It's rejected. Then we run the corrected program against those same values. It's accepted.

The worker's answer file no longer determines the result. The worker can change the proposed solution, while the controller owns the comparison.

That establishes the behavior for our five cases. A real application needs a much broader set of checks. This experiment is about who controls the checks. Five examples don't prove an application secure.

10. What our first test missed

09:00-10:30

Slow down for the first test. Then read the four rows in order.

There's another place to check: the step that releases the approved bytes. Our approval names the artifact and destination, and can be used once.

Our first substitution test was misleading. We published the approved artifact, then tried different bytes with the same approval. The gate refused. But the approval was already spent. That test couldn't tell us whether the byte check worked.

We changed the order. Wrong bytes first, while the approval was still unused. Then the wrong destination. Both were refused. The right bytes at the right destination succeeded, and only then did we test reuse.

Now each refusal tells us something. We also know the intended operation still works.

11. Where to look in your pipeline

10:30-11:30

Read the four questions. Pause after the fourth.

Start with one check your team relies on before a release. Find the expected answers, configuration, and helper code it uses. Then compare that list with what the agent can change.

Keep the candidate itself separate. It's supposed to be editable. You're looking for a way to change what counts as success without fixing the candidate.

Apply the same inspection to credentials, shared files, and uploads. For every denial, check that the intended operation still succeeds under the same conditions. That's how you know which control you actually tested.

12. Who controls the expected answer?

11:30-12:15

Point at the four lines. Say the last sentence, then stop.

Back to the original ticket. The program still returned 200. The worker couldn't overwrite the checker, but it could change the expected answer. That was enough to get a pass.

The fix let the worker keep editing the program while the controller kept the acceptance rules.

Before you trust a green check, find out whether the agent can change what makes it green. The code and recordings are here. Thank you.

Questions

13. What this experiment establishes

Questions

Use only in questions.

We measured whether these actions were allowed and whether the changed configuration stopped them. We didn't measure how often a model would try them. The five cases on the right are the whole policy. Passing them shows behavior on those five inputs and nothing broader.

14. The token reached another project

Questions

Use only in questions.

The service enforced a token restricted to the task. The intended input stayed readable. A shorter lifetime alone does not narrow permissions: a broad token can reach everything it covers for as long as it is valid. Scope and audience have to be enforced by the service that receives the token.

15. The shared file changed on the host

Questions

Use only in questions.

The change persisted in the next-job fixture on the host. We did not run a later job that consumed it, so that part is an implication, not a recording. The same write through a read-only bind was denied, and the worker's own output directory stayed writable.

16. The upload landed in another account

Questions

Use only in questions.

Reaching a service does not establish permission for every operation or recipient on it. The receiver recorded the stored payload, so this is delivery, not an attempted send. In the corrected path the broker sends one fixed report to the team account and refuses extra recipient, payload or URL fields. This is a loopback service model, not a tested firewall bypass.

17. A bug in our earlier release gate

Questions

Use only in questions.

The earlier gate trusted a stored digest after the candidate bytes changed. Reproducing it required direct access to the controller's object. We did not show a worker-to-controller exploit. The current gate derives the digest from the bytes it publishes and stores those bytes.

18. What we recorded

Questions

Use only in questions.

These counts describe checks, not vulnerability frequency. Each record has its own scope. The 49 are regression tests for the controller and the evidence validator, not isolation tests. The old gate audit is one reproduction of our own bug.

19. Run the experiment

Questions

Use only in questions.

The Linux experiments need a disposable Linux machine with bubblewrap, Python 3.10 or later, and make. TeX Live is only needed to rebuild the PDFs. On any operating system, make test and make factory run the local protocol checks without OS isolation. No cloud account or token is needed.

20. What the approval covers

Questions

Use only in questions.

The controller checks the outputs, then authenticates its decision together with the candidate, criteria, and destination bindings, using a local HMAC key. The approval names the task and the run, expires, and can be used once. It does not sign the raw observations. The full field list is in factory core.